COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	**CPE 010**	**SECTION:**	CPE21S4
DATE PERFORMED:	07 / 24 / 26	**DATE SUBMITTED:**	07 / 29 /26
NAME:	Mangcucang, Rhianne L.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 3.1 Linked List

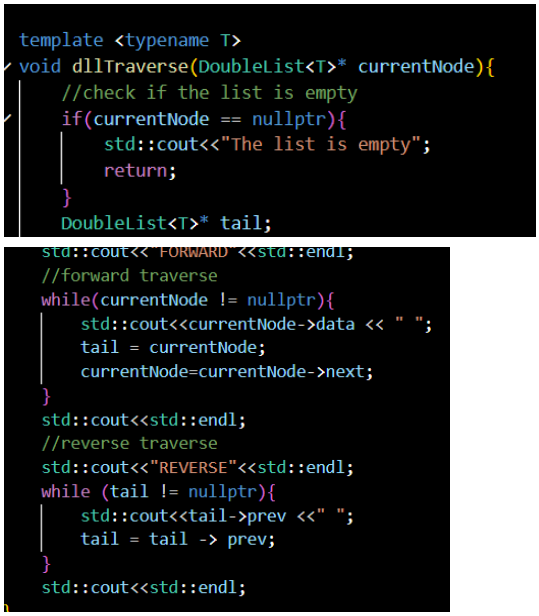
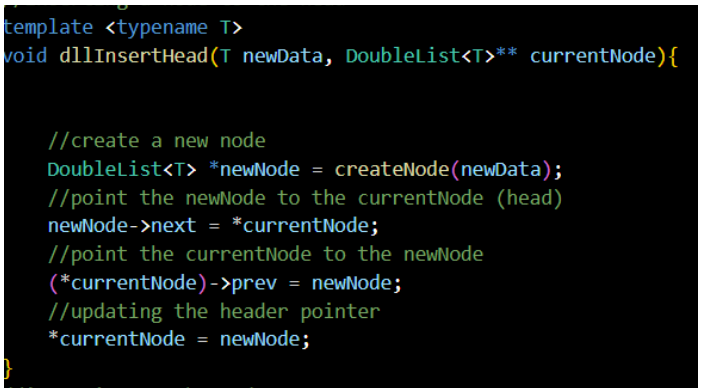
1. PROCEDURE OUTPUT

What procedures did you perform?

Screenshot	Testing the dllTraverse List FORWARD C P E REVERSE 0x7ff7bfe0b050 0x7ff7bfe0b030 0x7ff7bfe0b010 Testing the dllInserthead FORWARD X C P E REVERSE 0x7ff7bfe0b050 0x7ff7bfe0b030 0x7ff7bfe0b010 0x7ff7bfe0b070 Testing the dllInserthead FORWARD 0 X C P E REVERSE 0x7ff7bfe0b050 0x7ff7bfe0b030 0x7ff7bfe0b010 0x7ff7bfe0b070 0x7ff7bfe0b090 Testing the dllGenInsert FORWARD 0 X C A P E REVERSE 0x7ff7bfe0b050 0x7ff7bfe0b010 0x7ff7bfe0b0b0 0x7ff7bfe0b030 0x7ff7bfe0b070 0x7ff7bfe0b090 After Deleting FORWARD 0 X A P E REVERSE 0x7ff7bfe0b050 0x7ff7bfe0b010 0x7ff7bfe0b0b0 0x7ff7bfe0b070 0x7ff7bfe0b090

Discussion	The implementation shows the basic operations of a Doubly Linked List, such as initialization, forward and reverse traversal, node insertion, and node deletion. By using next and prev pointers, the list allows moving in both directions while updating the node links properly.

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot
Traversal	
Insertion at the head	

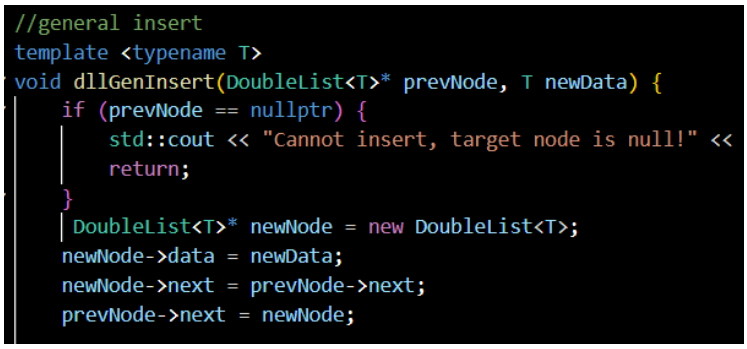
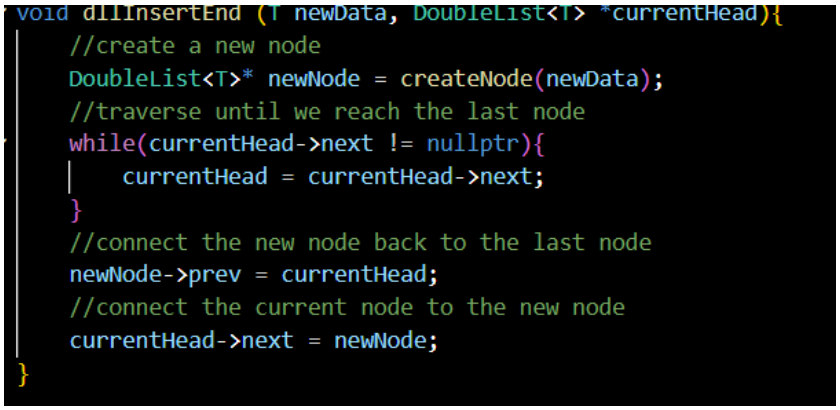
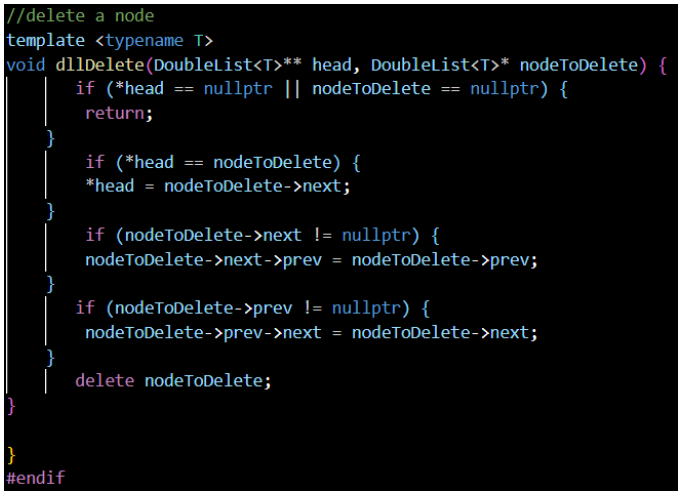
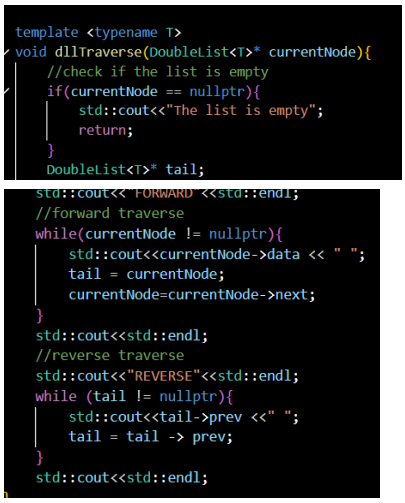
Insertion at any part of the list	
Insertion at the end	
Deletion of a node	

Table 3-2. Code for the List Operations

a.	Source Code	traverse(head);
	Console	CPE010
b.	Source Code	insertHead(&head, 'G');
	Console	GCPE010
c.	Source Code	insertAfter(secondNode, 'E'); // prevNode is 'P'
	Console	GCPEE010
d.	Source Code	deleteNode(&head, 'C');
	Console	GPEE010
e.	Source Code	deleteNode(&head, 'P');
	Console	GEE010
f.	Source Code	traverse(head);
	Console	GEE101

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshots	Analysis
	A doubly linked list uses two pointers: next and prev. The first loop goes FORWARD through the list using next and keeps track of the last item (tail). The second loop then uses tail->prev to step backward and print the list in REVERSE.

```
template <typename T>
void dllTraverse(DoubleList<T>* currentNode){
    //check if the list is empty
    if(currentNode == nullptr){
        std::cout<<"The list is empty";
        return;
    }
    DoubleList<T>* tail;
    std::cout<< "FORWARD" <<std::endl;
    //forward traverse
    while(currentNode != nullptr){
        std::cout<<currentNode->data << " ";
        tail = currentNode;
        currentNode=currentNode->next;
    }
    std::cout<<std::endl;
    //reverse traverse
    std::cout<<"REVERSE"<<std::endl;
    while (tail != nullptr){
        std::cout<<tail->prev <<" ";
        tail = tail -> prev;
    }
    std::cout<<std::endl;
```

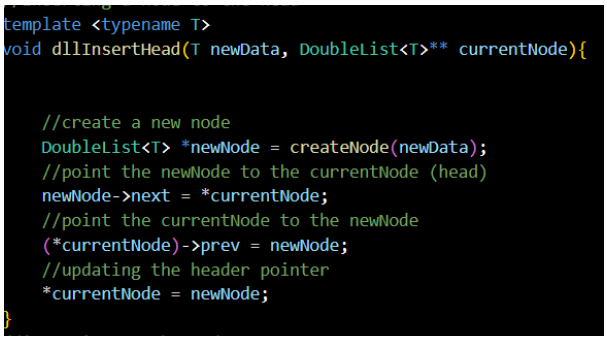
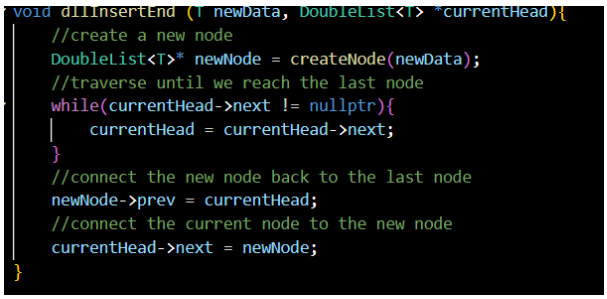
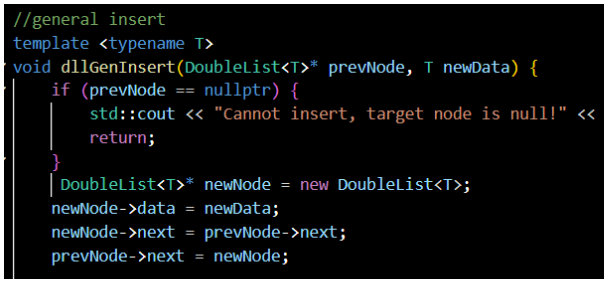
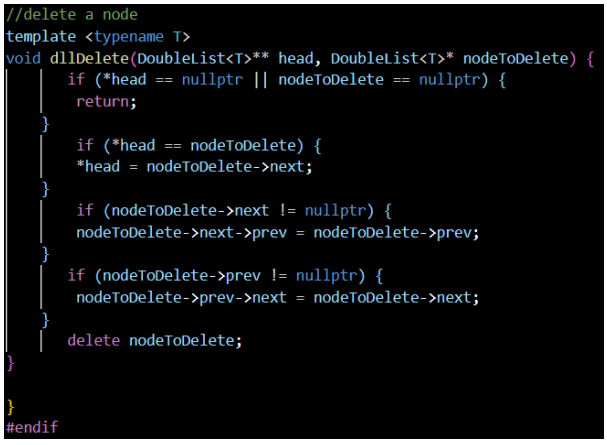
	To put a new item at the front, we connect the new node's next pointer to the old head. Then, we set the old head's prev pointer back to the new node so they point to each other. Finally, we update the head to be the new node.
	This code moves through the list until it finds the last node (next == nullptr). Once at the end, it links the new node's prev to the current last node and sets the last node's next to the new node.
	This adds a new node right after a specific node (prevNode). It sets the new node to point forward to the rest of the list and backward to prevNode, then updates prevNode->next to point to the new node
	Deleting a node in a doubly linked list is simple because we do not need to search from the start. We just reconnect the neighbor nodes around it using prev and next, then delete the target node to clear memory.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Node {
    string song;
    Node* next;
};

Node* head = NULL;
Node* tail = NULL;
Node* current = NULL;

// Create Playlist
void createPlaylist() {
    int n;
    string title;

    cout << "How many songs? ";
    cin >> n;
    cin.ignore();

    for (int i = 1; i <= n; i++) {
        cout << "Enter Song " << i << ": ";
        getline(cin, title);

        Node* newNode = new Node;
        newNode->song = title;

        if (head == NULL) {
            head = tail = newNode;
            tail->next = head;
        } else {
            tail->next = newNode;
            tail = newNode;
            tail->next = head;
        }
```

```
    }

    current = head;
    cout << "Playlist created successfully!\n";
}

// Add Song
void addSong(string title) {
    Node* newNode = new Node;
    newNode->song = title;

    if (head == NULL) {
        head = tail = newNode;
        tail->next = head;
        current = head;
    } else {
        tail->next = newNode;
        tail = newNode;
        tail->next = head;
    }

    cout << "Song added!\n";
}

// Display Playlist
void displayPlaylist() {
    if (head == NULL) {
        cout << "Playlist is empty.\n";
        return;
    }

    Node* temp = head;

    cout << "\nPlaylist:\n";
    do {
        cout << "- " << temp->song << endl;
```

```
        temp = temp->next;
    } while (temp != head);
}

// Remove Song
void removeSong(string title) {
    if (head == NULL) {
        cout << "Playlist is empty.\n";
        return;
    }

    Node *temp = head, *prev = tail;

    do {
        if (temp->song == title) {

            if (head == tail) {
                delete temp;
                head = tail = current = NULL;
            }
            else if (temp == head) {
                head = head->next;
                tail->next = head;
                if (current == temp)
                    current = head;
                delete temp;
            }
            else if (temp == tail) {
                tail = prev;
                tail->next = head;
                if (current == temp)
                    current = head;
                delete temp;
            }
            else {
                prev->next = temp->next;
```

```
            if (current == temp)
                current = temp->next;
            delete temp;
        }

        cout << "Song removed!\n";
        return;
    }

    prev = temp;
    temp = temp->next;

} while (temp != head);

cout << "Song not found!\n";
}

// Next Song
void nextSong() {
    if (current == NULL) {
        cout << "Playlist is empty.\n";
        return;
    }

    current = current->next;
    cout << "Now Playing: " << current->song << endl;
}

// Previous Song
void previousSong() {
    if (current == NULL) {
        cout << "Playlist is empty.\n";
        return;
    }

    Node* temp = head;
```

```
    while (temp->next != current)
        temp = temp->next;

    current = temp;

    cout << "Now Playing: " << current->song << endl;
}

// Play Songs in Loop
void playSongsLoop() {
    if (head == NULL) {
        cout << "Playlist is empty.\n";
        return;
    }

    char choice;

    do {
        cout << "\nNow Playing: " << current->song << endl;
        cout << "Press N for Next Song | Q to Stop: ";
        cin >> choice;

        if (choice == 'N' || choice == 'n') {
            current = current->next;
        }

    } while (choice != 'Q' && choice != 'q');
}

int main() {

    int choice;
    string song;

    do {
```

```
cout << "\n=== SONG PLAYLIST ===\n";
cout << "1. Create Playlist\n";
cout << "2. Add Song\n";
cout << "3. Remove Song\n";
cout << "4. Display Playlist\n";
cout << "5. Next Song\n";
cout << "6. Previous Song\n";
cout << "7. Play Songs in Loop\n";
cout << "8. Exit\n";
cout << "Choice: ";
cin >> choice;
cin.ignore();

switch(choice) {

case 1:
    createPlaylist();
    break;

case 2:
    cout << "Song title: ";
    getline(cin, song);
    addSong(song);
    break;

case 3:
    cout << "Song to remove: ";
    getline(cin, song);
    removeSong(song);
    break;

case 4:
    displayPlaylist();
    break;
```

```
        case 5:
            nextSong();
            break;

        case 6:
            previousSong();
            break;

        case 7:
            playSongsLoop();
            break;

        case 8:
            cout << "Goodbye!\n";
            break;

        default:
            cout << "Invalid choice!\n";
        }

    } while(choice != 8);

    return 0;
}
```

Conclusion

In this activity, we learned how Singly, Doubly, and Circular Linked Lists work in C++ by seeing how nodes use next and prev pointers to connect items dynamically in memory instead of using fixed arrays. The procedure highlighted that while singly linked lists only move forward, doubly linked lists allow moving both forward and backward, requiring careful pointer updates during insertion and deletion to prevent broken links. This concept was applied in the supplementary activity, where connecting the last song back to the first created a Circular Doubly Linked List that enabled smooth playlist navigation using nextSong() and prevSong() without NULL errors. This lab made linked list concepts much easier to understand through coding, though I can still practice handling edge cases safely, such as deleting the final remaining node in a circular list.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, expl concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.